

Globbering Wildcards - Angabe von Dateien in der Shell * bel. viele Zeichen ? genau 1 Zeichen [abc] Zeichen aus Menge *.txt Dateien mit der Endung .txt	Regex ^ Zeilenanfang \$ Zeilenende ? Optional (0 oder 1 Mal) * Beliebig oft (auch 0 Mal) + Mindestens 1 Mal {n} Genau n-mal {n,} Mindestens n-mal {n,m} Zwischen n- und m-mal {0,m} Maximal m-mal \ Maskiert Sonderzeichen (wird Text) . [] Zeichenklassen (beliebig / Auswahl) & Backreference (Rückreferenz) () Gruppe bilden Alternative
Allgemeine Regeln 'Text \$HOME' Kommandos/Vars nicht ausgewertet "Text \$HOME" wird ausgewertet \$(kommando) Kommando ausgef., Ausgabe einf. \ escape	Kurzformen [-f "\$datei"] && echo "Datei vorhanden"
Spezielle Parameter \$* / \$@ Stellungsparameter (\$1, \$2 ... \$@) \$@ Anzahl Stellungsparameter \$? letzter Rückgabewert \$\$ PID der Shell \$0 Namen des Skripts	Aufbau Shellskript #!/bin/bash
Optionen für [] oder test	If Anweisung
Zahlen: -eq -ne gleich / ungleich -lt -le kleiner / kleiner oder gleich -gt -ge größer / größer oder gleich	if [Bedingung] then ---kommandos--- elif [Bedingung2] then ---kommandos--- else ---kommandos--- fi
Strings: = != gleich / ungleich -z / -n leer/nicht leer	
Dateien: -e existiert -f / -d Datei / Verzeichnis -r -w -x lesbar/schreibbar/ausführbar -s existiert und nicht leer	Allgemein: -a AND -o OR z.B.: if [-e "\$datei" -a -f "\$datei"] ; then ...
for-Schleife	while-Schleife
for datei in *.txt do echo "Bearbeite \$datei" done for arguments #Liste der Argumente do echo "Parameter: \$arg" # Gibt nacheinander # A, B und C aus done	while read line do kommandos done < file.txt while read spalte0 spalte1 // Trennsymb.: IFS do kommandos done < file.txt

grep grep [-Optionen] regex [dateien] -c (count) Gibt Anzahl passender Zeilen aus -h (hide) Verbirgt Dateinamen in der Ausgabe -i (ignore) Ignoriert Groß-/Kleinschreibung -l (list) Zeigt nur Dateinamen mit Treffern -n (number) Zeigt Zeilennummern an -o (only) Zeigt nur exakten Match pro Zeile -s (suppress) Unterdrückt Fehlermeldungen -v (vice versa) Invertiert Suche (zeigt Nicht-Treffer) -E (extended) Aktiviert ERE (extended Regex) -F (fast) Sucht nach festem Text (ohne Regex) Note: Regexp sollte in '' stehen, damit die Shell \$, *, [oder nicht selbst interpretiert. Wenn man das aber braucht, dann "" Leerzeichen/tabs bei Sed und Grep ist [[:space:]] oder \s oder ' ', nur Tab ist \t	sed substituete sed -E '/pattern/s/regex/replace/flags' mydata Bsp.: 's/alt/neu/g' alle Vorkommen pro Zeile ersetzen, pattern ist optional, d.h. auch: 's/regex/replace/flags' ODER: '1,4 s/regex/replace/flags' -- Ersetze nur in Zeile 1-4 Flags für substitute: g (global) Ersetzt alle Treffer pro Zeile n (number) Ersetzt nur das n-te Vorkommen p (print) Gibt geänderte Zeile aus i (ignore) Ignoriert Groß-/Kleinschreibung delete '/pattern/d' Zeilen löschen die Pattern enthalten '1,4 d' Zeilen 1-4 löschen														
cut - Spaltenauswahl cut -d ';' -f 3 oder -c 10 -d: delimiter -f: field (Spalte) -c: character (Zeichen)	sort <table> <tr> <th>Struktur</th><th>Beschreibung</th></tr> <tr> <td>sort datei</td><td>Alphabetisch (A-Z)</td></tr> <tr> <td>sort -n datei</td><td>Numerisch (mathematischer Wert)</td></tr> <tr> <td>sort -r datei</td><td>Rückwärts (Z-A / groß nach klein)</td></tr> <tr> <td>sort -u datei</td><td>Löscht Duplikate (Unique)</td></tr> <tr> <td>sort -t';' -k2,2n datei</td><td>Spalte 2 numerisch mit Trenner ';' </td></tr> </table>	Struktur	Beschreibung	sort datei	Alphabetisch (A-Z)	sort -n datei	Numerisch (mathematischer Wert)	sort -r datei	Rückwärts (Z-A / groß nach klein)	sort -u datei	Löscht Duplikate (Unique)	sort -t';' -k2,2n datei	Spalte 2 numerisch mit Trenner ';'		
Struktur	Beschreibung														
sort datei	Alphabetisch (A-Z)														
sort -n datei	Numerisch (mathematischer Wert)														
sort -r datei	Rückwärts (Z-A / groß nach klein)														
sort -u datei	Löscht Duplikate (Unique)														
sort -t';' -k2,2n datei	Spalte 2 numerisch mit Trenner ';'														
general commands head -n5 d Erste 5 Zeilen anzeigen tail -n1 d Letzte Zeile anzeigen wc -l < d Zeilen zählen (Zeilenumbrüche) wc -w d Wörter zählen wc -c < d Bytes zählen	Zeichen ersetzen/löschen (tr) Syntax: cat datei tr [Optionen] 'Suchen' 'Ersetzen' (Arbeitet Zeichen für Zeichen) <table> <tr> <th>Struktur</th><th>Beschreibung</th></tr> <tr> <td>tr 'a-z' 'A-Z'</td><td>Klein- to Großbuchstaben</td></tr> <tr> <td>tr '\t' ' '</td><td>Tabs to Leerzeichen</td></tr> <tr> <td>tr -d '0-9'</td><td>Alle Zahlen löschen (delete)</td></tr> <tr> <td>tr -s ' '</td><td>Mehrfache Leerzeichen zu einem stauchen (squeeze)</td></tr> </table>	Struktur	Beschreibung	tr 'a-z' 'A-Z'	Klein- to Großbuchstaben	tr '\t' ' '	Tabs to Leerzeichen	tr -d '0-9'	Alle Zahlen löschen (delete)	tr -s ' '	Mehrfache Leerzeichen zu einem stauchen (squeeze)				
Struktur	Beschreibung														
tr 'a-z' 'A-Z'	Klein- to Großbuchstaben														
tr '\t' ' '	Tabs to Leerzeichen														
tr -d '0-9'	Alle Zahlen löschen (delete)														
tr -s ' '	Mehrfache Leerzeichen zu einem stauchen (squeeze)														
<table> <tr> <th>Struktur</th><th>Beschreibung</th></tr> <tr> <td>< d</td><td>Datei als Eingabe (stdin)</td></tr> <tr> <td>> d</td><td>Ausgabe neu / Überschreiben (stdout)</td></tr> <tr> <td>>> d</td><td>Ausgabe unten anhängen (stdout)</td></tr> <tr> <td>2> d</td><td>Nur Fehlermeldungen umleiten (stderr)</td></tr> <tr> <td>2>&1</td><td>stderr auf dasselbe Ziel wie stdout</td></tr> <tr> <td>> /dev/null</td><td>Ausgabe komplett verwerfen</td></tr> </table>	Struktur	Beschreibung	< d	Datei als Eingabe (stdin)	> d	Ausgabe neu / Überschreiben (stdout)	>> d	Ausgabe unten anhängen (stdout)	2> d	Nur Fehlermeldungen umleiten (stderr)	2>&1	stderr auf dasselbe Ziel wie stdout	> /dev/null	Ausgabe komplett verwerfen	Beispiel <pre>sort < namen.txt grep 'ERROR' log > fehler.txt echo 'fertig' >> protokoll.txt ls /nichtda 2> fehler.log befehl > alles.log 2>&1 befehl > /dev/null</pre>
Struktur	Beschreibung														
< d	Datei als Eingabe (stdin)														
> d	Ausgabe neu / Überschreiben (stdout)														
>> d	Ausgabe unten anhängen (stdout)														
2> d	Nur Fehlermeldungen umleiten (stderr)														
2>&1	stderr auf dasselbe Ziel wie stdout														
> /dev/null	Ausgabe komplett verwerfen														
Arithm. Ausdrücke berechnen \$((arith.Ausdruck))															